

Oefeningen

Deze oefeningen richten zich op de fundamenteën: het herkennen van de 3-stappenmethode en het opstellen van simpele, sequentiële algoritmen.

Tips voor het oplossen van de opdrachten

Voordat je begint met schrijven, is het cruciaal om eerst na te denken. Volg de **3-stappenmethode** uit de theorie om elk probleem systematisch aan te pakken. Dit zorgt ervoor dat je de logica en het proces begrijpt, wat veel belangrijker is dan direct een oplossing te bedenken.

Stap 1: Probleemanalyse

Beschrijving: Begin met het begrijpen van het probleem in de kern. Stel jezelf de vraag: "Wat is het echte doel van deze opdracht?". Formuleer het antwoord in je eigen, simpele woorden. Dit helpt je om alle overbodige details te filteren en je te focussen op de essentie.

Hoe doe je dit: Schrijf een korte, duidelijke zin die het doel van de opdracht samenvat, alsof je het uitlegt aan iemand die de opdracht niet heeft gelezen.

Stap 2: Identificeer input, verwerking & output

Beschrijving: Dit is het 'keukenwerk' van je algoritme. Bedenk welke ingrediënten (input) je nodig hebt, welke handelingen (verwerking) je daarop moet uitvoeren en wat het eindresultaat (output) moet zijn. Dit dwingt je om alle onderdelen van je oplossing in kaart te brengen voordat je de stappen uitschrijft.

Hoe doe je dit: Maak een lijstje met drie duidelijke punten:

- **Input:** Wat heb ik nodig om te starten? (bijv. een getal, een naam, een keuze)
- **Verwerking:** Welke stappen, berekeningen of keuzes moet ik maken?
- **Output:** Wat is het uiteindelijke, gewenste resultaat? (bijv. een berekende prijs, een bericht)

Stap 3: Schrijf de pseudocode

Beschrijving: Nu je alle stappen hebt geanalyseerd en opgesomd, vertaal je je plan naar een stappenlijst in duidelijke, menselijke taal. Gebruik de kernconcepten zoals ALS...DAN voor keuzes en ZOLANG...HERHAAL voor herhalingen. De pseudocode is de blauwdruk van je uiteindelijke programma en moet zo helder zijn dat elke "domme assistent" het kan uitvoeren.

Hoe doe je dit: Gebruik een genummerde lijst en schrijf elke stap op een nieuwe regel. Zorg dat je instructies ondubbelzinnig zijn.

Door deze aanpak te volgen, leer je niet alleen de opdrachten op te lossen, maar ontwikkel je ook de essentiële vaardigheden van een programmeur: probleemoplossend en logisch denken.

Basisprincipes

Deze oefeningen richten zich op de fundamenteën: het herkennen van de 3-stappenmethode en het opstellen van simpele, sequentiële algoritmen.

Een T-shirt wassen

Opdracht: Je wilt je favoriete T-shirt wassen met de hand. Beschrijf de stappen die je moet nemen om dit te doen, van begin tot einde, in duidelijke pseudocode.

▼ Oplossing

Stap 1. Probleemanalyse: Ik wil een vuil T-shirt schoon krijgen door het handmatig te wassen.

Stap 2. Input/Verwerking/Output:

Input: Een vuil T-shirt, water, zeep.

Verwerking: De handelingen van het shirt natmaken, inzepen, wrijven en spoelen.

Output: Een schoon, nat T-shirt.

Stap 3. Pseudocode:

PAK het vuile T-shirt.

VUL een wasbak of emmer met water.

VOEG zeep toe aan het water.

PLAATS het T-shirt in het zeepwater.

WRIJF over het T-shirt om het schoon te maken.

HAAL het T-shirt uit het zeepwater.

SPOEL het T-shirt grondig uit onder stromend water totdat alle zeepresten verdwenen zijn.

WRING het T-shirt voorzichtig uit.

Vrienden uitnodigen

Opdracht: Je wilt een vriend uitnodigen om samen een film te kijken. Schrijf een algoritme in pseudocode dat dit proces beschrijft. Denk na over de verschillende manieren waarop je contact kunt opnemen en welke stappen daarop volgen.

▼ Oplossing

Stap 1. Probleemanalyse: Ik wil een vriend uitnodigen om samen een film te kijken en te bepalen waar en wanneer we dat gaan doen.

Stap 2. Input/Verwerking/Output:

Input: Een vriend, een communicatiemiddel (telefoon, chat-app), een mogelijke film.

Verwerking: Het versturen van het bericht, wachten op een antwoord en het plannen van de activiteit.

Output: Een overeengekomen plan voor de filmavond.

Stap 3. Pseudocode:

KIES een vriend die je wilt uitnodigen.

OPEN je chat-app of bel de vriend op.

STUUR een bericht met de vraag of hij/zij zin heeft om een film te kijken.

WACHT op een antwoord.

ALS de vriend ja antwoordt:

STEL een paar mogelijke datums en tijden voor.

ALS jullie een datum en tijd afspreken:

STUUR een bevestiging van het plan.

ANDERS (geen overeenkomst):

SLUIT de chat.

ANDERS (als de vriend nee antwoordt):

BEDANK de vriend en zeg dat je het een andere keer probeert.

SLUIT de chat.

Condities en variabelen

Deze oefeningen vereisen het gebruik van **condities** (als-dan) en **variabelen** (doosjes met een etiket).

De parkeerautomaat

Opdracht: Schrijf een algoritme voor een parkeerautomaat. De automaat accepteert enkel muntstukken van 0.50 euro, 1 euro en 2 euro. De parkeerkosten zijn 1.50 euro per uur. Je hoeft nog geen wisselgeld te geven.

▼ **Oplossing:**

Stap 1. Probleemanalyse: De automaat moet het juiste bedrag incasseren voor een uur parkeren.

Stap 2. Input/Verwerking/Output:

Input: Ingeworpen muntstukken.

Verwerking: Het optellen van de ingeworpen bedragen en vergelijken met het vereiste bedrag.

Output: Een parkeerticket.

Stap 3. Pseudocode:

MAAK een variabele totaalBetaald en zet de waarde op 0.

BEPAAI de vereiste parkeerKosten als 1.50 euro.

VRAAG de gebruiker om muntstukken in te werpen.

ZOLANG de gebruiker een muntstuk inwerpt:

NEEM het muntstuk.

TEL de waarde van het muntstuk op bij totaalBetaald.

ALS totaalBetaald groter is dan of gelijk is aan parkeerKosten:

DRUK een parkeerticket af.

STOP het proces.

ANDERS (als het bedrag nog te laag is):

GEEF een bericht dat er nog munten nodig zijn.

Opdracht: Schrijf een algoritme voor een slimme wekker. De wekker heeft een `alarmTijd` en een variabele `isWakker`. Het alarm moet pas afgaan als het `alarmTijd` is en de variabele `isWakker` onwaar (false) is. Als de gebruiker op de snooze-knop drukt, moet het alarm over 9 minuten opnieuw afgaan.

▼ **Oplossing:**

Stap 1. Probleemanalyse: Een wekker die alleen afgaat als de gebruiker nog slaapt en die een snooze-functie heeft.

Stap 2. Input/Verwerking/Output:

Input: De huidige tijd, de `alarmTijd`, de knop snooze.

Verwerking: Tijd vergelijken en de snooze-functie uitvoeren.

Output: Een geluidssignaal.

Stap 3. Pseudocode:

MAAK een variabele `isWakker` en zet de waarde op `ONWAAR`.

MAAK een variabele `alarmTijd` en stel deze in op `07:00`.

ZOLANG de huidige tijd niet gelijk is aan `alarmTijd`:

WACHT.

ALS de huidige tijd gelijk is aan `alarmTijd` EN `isWakker` is `ONWAAR`:

SPEEL het alarmgeluid af.

WACHT 30 seconden.

ALS de gebruiker op de snooze-knop drukt:

VERHOOG de `alarmTijd` met 9 minuten.

```
STOP het afspelen van het alarmgeluid.
```

```
GA TERUG naar stap 3 (de ZOLANG-loop).
```

```
ANDERS (als de gebruiker niet op snooze drukt):
```

```
STOP het afspelen van het alarmgeluid.
```

```
ZET isWakker op WAAR.
```

Gecombineerde logica

Deze oefeningen combineren alle geleerde concepten en vereisen complexere logica.

De Pizza-bestelautomaat

Opdracht: Schrijf een algoritme voor een pizza-bestelautomaat. De automaat moet het volgende kunnen:

- De gebruiker kan kiezen uit een 'Margherita' of een 'Funghi'.
- De prijs van een Margherita is €10.
- De prijs van een Funghi is €12.
- De gebruiker kan een extra topping toevoegen (€2 extra).
- De automaat moet de totale prijs berekenen en tonen.
- De betaling verloopt cash. Als de gebruiker minder geld geeft, moet de automaat een foutmelding geven.
- Als de gebruiker exact of meer geld geeft, moet de automaat de bestelling bevestigen en eventueel wisselgeld berekenen.

▼ **Oplossing:**

Stap 1. Probleemanalyse: Een algoritme dat een pizza-bestelling verwerkt, inclusief keuzemogelijkheden voor type en topping, en vervolgens de betaling afhandelt.

Stap 2. Input/Verwerking/Output:

Input: Keuze voor pizza, keuze voor topping, ingeworpen geld.

Verwerking: Bepalen van de prijs op basis van de keuzes, optellen van de extra kosten, berekenen van het wisselgeld.

Output: De totale prijs, een bestellingsbevestiging, eventueel wisselgeld.

Stap 3. Pseudocode:

MAAK een variabele totalePrijs en zet de waarde op 0.

VRAAG de gebruiker: "Kies een pizza: 1) Margherita (€10) of 2) Funghi (€12)?"

LEES de keuze van de gebruiker.

ALS de keuze 1 is:

ZET totalePrijs op 10.

ANDERS ALS de keuze 2 is:

ZET totalePrijs op 12.

ANDERS (ongeldige keuze):

TOON een foutmelding en begin opnieuw.

VRAAG de gebruiker: "Wilt u een extra topping? (ja/nee) (€2 extra)"

LEES de keuze van de gebruiker.

ALS de keuze "ja" is:

```
TEL 2 op bij totalePrijs.
```

```
TOON de totalePrijs aan de gebruiker.
```

```
MAAK een variabele betaaldBedrag en zet de waarde op 0.
```

```
VRAAG de gebruiker om het geld in te werpen totdat  
betaaldBedrag groter is dan of gelijk is aan totalePrijs.
```

```
ALS betaaldBedrag kleiner is dan totalePrijs:
```

```
TOON een foutmelding: "Onvoldoende betaald. Begin  
opnieuw."
```

```
ANDERS (betaling is succesvol):
```

```
BEREKEN wisselgeld als betaaldBedrag min totalePrijs.
```

```
TOON een bestellingsbevestiging: "Bestelling  
ontvangen. Uw wisselgeld is: " + wisselgeld.
```

```
GEEF het berekende wisselgeld terug.
```

De Bibliotheekrobot

Opdracht: Een robot moet een boek vinden in een bibliotheek. Je hebt een lijst met boekenplanken, elk met een uniek nummer, beginnende vanaf 1.

Je hebt ook een variabele `gezochtBoek` met de titel die je zoekt.

De robot kan alleen per boekenplank controleren.

Schrijf een algoritme in pseudocode dat de robot instrueert om het boek te vinden en dan terug te keren naar het beginpunt.

Het stappenplan moet de robot ook stoppen zodra hij het boek heeft gevonden, zelfs als hij nog niet alle planken heeft doorzocht.

▼ Oplossing:

Stap 1. Probleemanalyse: Een robot moet systematisch een lijst doorlopen totdat hij een specifieke taak heeft voltooid, en dan stoppen en terugkeren.

Stap 2. Input/Verwerking/Output:

Input: Een lijst met boekenplanken, de titel van het gezochtBoek.

Verwerking: Het doorlopen van de lijst, het controleren van elke plank en het stoppen van het proces als het boek is gevonden.

Output: De locatie van het boek.

Stap 3. Pseudocode:

MAAK een variabele gezochtBoek met de waarde "De Gids voor de Galaxis".

MAAK een variabele gevonden en zet de waarde op ONWAAR.

BEPAAAL de huidigePlank en zet de waarde op 1.

ZOLANG gevonden ONWAAR is:

GA naar huidigePlank.

CONTROLEER alle boeken op deze plank.

ALS het gezochtBoek op deze plank ligt:

TOON een bericht: "Boek gevonden op plank " + huidigePlank.

ZET gevonden op WAAR.

ANDERS:

VERHOOG huidigePlank met 1.

ALS huidigePlank het einde van de bibliotheek heeft bereikt (en het boek nog niet is gevonden):

TOON een bericht: "Boek niet gevonden."

STOP het zoeken.

GA TERUG naar het beginpunt van de bibliotheek.

Blockly

We gaan verder met Blockly (<https://blockly.games>) zodat je de algoritmes en het logisch denken verder kan toepassen. Maak zeker gebruik van de 3-stappenmethode wanneer de opdrachten iets moeilijker worden.

- Muziek – Maak muziek met herhaling en timing.
- Leervijver – Programmeer een strategie met condities.
- Vijver – Schrijf jouw eerste echte JavaScript-code om te winnen van tegenstanders.

Scratch-oefeningen

[Link naar de Scratch-editor](#)

In dit hoofdstuk werk je stap voor stap aan Scratch-oefeningen die oplopen in moeilijkheidsgraad.

Elke oefening bevat een duidelijke vraagstelling, een lijst met functionaliteiten, en een overzicht van de Scratch-blokken die je kan gebruiken.

Het is de bedoeling dat je initieel **Functionaliteiten** en **Hulp: Stappen & Scratch-blokken** nog **NIET** meteen opent.

De bedoeling van deze oefeningen is dat je zelf leert nadenken over wat je allemaal nodig hebt om een bepaald probleem op te lossen en dat je nadenkt over welke stappen je in welke volgorde moet uitvoeren om het probleem op te lossen.

Eens je de vraagstelling gelezen hebt, probeer je eerst zelf eens een stappenplan uit te schrijven om de taak te kunnen voltooien. Nadien kan je ter verificatie naar **functionaliteiten** kijken om te controleren of je niets over het hoofd hebt gezien.

Zit je echt volledig vast met een oefening dan kun je de sectie **Hulp: Stappen & Scratch-blokken** raadplegen. Maar dit is eigenlijk het antwoord van de oefening. Als je hier meteen gaat kijken zonder zelf nagedacht te hebben over de stappen zal je niet veel leren van de oefening. Kijk hier dus alleen naar als allerlaatste optie.

Persoonlijke begroeting

Moeilijkheidsgraad: Beginner

Vraagstelling

Vraag de gebruiker naar zijn of haar naam en toon daarna een persoonlijke begroeting op het scherm.

▼ Functionaliteiten

- Vraag de naam van de gebruiker.
- Sla de ingevoerde naam op in een variabele.
- Toon de tekst "Welkom, [naam]!" op het scherm.

▼ Hulp: Stappen & Scratch-blokken

- **vraag [Wat is je naam?] en wacht** (Sensing: "ask [] and wait")
- **antwoord** (Sensing: "answer")

- `maak variabele [naam]` (Variables: "Make a Variable")
- `stel [naam] in op (antwoord)` (Variables: "set [naam] to []")
- `zeg [Welkom, (naam)!] gedurende [2] seconden` (Looks: "say [] for [] seconds")

Leeftijdscontrole

Moeilijkheidsgraad: Beginner

Vraagstelling

Vraag de bezoeker om zijn of haar leeftijd in te geven. Als de leeftijd 18 of ouder is, toon een leuke boodschap en verander de achtergrond naar iets feestelijks. Als de leeftijd jonger is dan 18, toon dan de boodschap "Sorry, je mag niet binnen." en verander de achtergrond naar iets saaiers.

▼ Functionaliteiten

- Vraag de leeftijd op en sla deze op in een variabele.
- Vergelijk: is de leeftijd groter of gelijk aan 18.
- Toon boodschap en verander achtergrond afhankelijk van de voorwaarde.

▼ Hulp: Stappen & Scratch-blokken

- `vraag [Hoe oud ben je?] en wacht` (Sensing: "ask [] and wait")
- `antwoord` (Sensing: "answer")
- `maak variabele [leeftijd]` (Variables: "Make a Variable")
- `stel [leeftijd] in op (antwoord)` (Variables: "set [leeftijd] to []")
- `als <(leeftijd) ≥ 18> dan ... anders ...` (Control: "if <> then else")
- `zeg [Welkom! Je bent volwassen] gedurende [2] seconden` (Looks: "say [] for [] seconds")

- `schakel achtergrond naar [feest]` (Looks: "switch backdrop to []")

Highscore-systeem

Moeilijkheidsgraad: Gemiddeld

Vraagstelling

Maak een klik-spelletje waarin de speler een score kan halen. Vergelijk deze score met de hoogste score tot nu toe en toon of er een nieuwe highscore is.

▼ Functionaliteiten

- Maak variabelen score en highscore.
- Score verhoogt bij klikken op sprite.
- Na afloop: vergelijk score met highscore.
- Indien hoger: update highscore en toon melding.

▼ Hulp: Stappen & Scratch-blokken

- `maak variabele [score]` en `[highscore]` (Variables: "Make a Variable")
- `wanneer op deze sprite geklikt` (Events: "when this sprite clicked")
- `verander [score] met 1` (Variables: "change [score] by []")
- `als <(score) > (highscore)> dan stel [highscore] in op (score)`
(Control: "if <> then", Variables: "set [highscore] to []")
- `zeg [Nieuwe highscore!] gedurende [2] seconden` (Looks: "say [] for [] seconds")

Formulier-validatie

Moeilijkheidsgraad: Uitdagend

Vraagstelling

Simuleer een formulier waarin de speler naam, leeftijd en e-mail moet invullen. Controleer of de leeftijd minstens 18 is en of de e-mail een "@" bevat. Toon "Formulier ok" of "Niet geldig".

▼ Functionaliteiten

- Vraag naam, leeftijd en e-mail en sla op in variabelen.
- Controleer leeftijd ≥ 18 .
- Controleer of e-mail een "@" bevat.
- Toon resultaat afhankelijk van de validatie.

▼ Hulp: Stappen & Scratch-blokken

- `vraag [Wat is je naam?] en wacht` (Sensing: "ask [] and wait")
- `vraag [Wat is je leeftijd?] en wacht` (Sensing: "ask [] and wait")
- `vraag [Wat is je e-mail?] en wacht` (Sensing: "ask [] and wait")
- `maak variabele [naam] [leeftijd] [email]` (Variables: "Make a Variable")
- `stel [leeftijd] in op (antwoord)` (Variables: "set [leeftijd] to []")
- `als <(leeftijd)  $\geq$  18> dan ...` (Control: "if <> then")
- `Controle van "@" in e-mail` (Operators: "x contains y")
- `zeg [Formulier ok] of zeg [Niet geldig]` (Looks: "say []")

Rekenmachine

Moeilijkheidsgraad: Gemiddeld

Vraagstelling

Vraag de gebruiker om twee getallen en een bewerking (optellen of aftrekken). Geef het resultaat terug.

▼ Functionaliteiten

- Vraag twee getallen en sla ze op.
- Vraag naar de gewenste bewerking.
- Voer de berekening uit afhankelijk van het antwoord.
- Toon het resultaat.

▼ Hulp: Stappen & Scratch-blokken

- `vraag [Geef het eerste getal] en wacht` (Sensing: "ask [] and wait")
- `stel [getal1] in op (antwoord)` (Variables: "set [getal1] to []")
- `vraag [Geef het tweede getal] en wacht` (Sensing: "ask [] and wait")
- `stel [getal2] in op (antwoord)` (Variables: "set [getal2] to []")
- `vraag [Wil je optellen of aftrekken?] en wacht` (Sensing: "ask [] and wait")
- `als <(antwoord) = [optellen]> dan stel [resultaat] in op ((getal1) + (getal2)) anders stel [resultaat] in op ((getal1) - (getal2))` (Control: "if <> then else", Operators: "+" en "-")
- `zeg [Het resultaat is (resultaat)]` (Looks: "say []")

Quiz met meerdere vragen

Moeilijkheidsgraad: Gemiddeld

Vraagstelling

Maak een quiz met drie vragen. Hou de score bij en toon op het einde het resultaat.

▼ Functionaliteiten

- Maak een variabele score.
- Stel drie vragen.
- Controleer telkens het antwoord en verhoog score bij juist.
- Toon eindscore.

▼ Hulp: Stappen & Scratch-blokken

- `maak variabele [score]` (Variables: "Make a Variable")
- `vraag [Vraag 1: ...] en wacht` (Sensing: "ask [] and wait")
- `als <(antwoord) = [juist]> dan verander [score] met 1` (Control: "if <answer = [correct answer]> then", Variables: "change [score] by 1")
- `zeg [Je score is (score)]` (Looks: "say []")

Login-systeem met 2 pogingen

Moeilijkheidsgraad: Gemiddeld

Vraagstelling

Maak een login-systeem dat een wachtwoord vraagt. Bij juiste invoer krijgt de speler toegang. Bij twee foute pogingen verschijnt "Toegang geblokkeerd".

▼ Functionaliteiten

- Maak variabele pogingen en stel in op 0.
- Vraag het wachtwoord en controleer dit.
- Verhoog pogingen als fout.
- Toon boodschap bij succes of blokkering.

▼ Hulp: Stappen & Scratch-blokken

- `maak variabele [pogingen]` (Variables: "Make a Variable")
- `stel [pogingen] in op [0]` (Variables: "set [pogingen] to [0]")
- `vraag [Wat is het wachtwoord?] en wacht` (Sensing: "ask [] and wait")
- `als <(antwoord) = [geheim]> dan zeg [Welkom!] anders verander [pogingen] met 1` (Control: "if <> then else", Variables: "change [pogingen] by [1]")
- `als <(pogingen) = 2> dan zeg [Toegang geblokkeerd]` (Control: "if <> then")

Animatie door toetsdruk

Moeilijkheidsgraad: Gemiddeld

Vraagstelling

Maak een sprite die links en rechts beweegt met toetsen A en D en springt met spatie. Laat bij elke actie ook een visueel effect verschijnen.

▼ Functionaliteiten

- Beweeg links en rechts met toetsen A en D.
- Laat de sprite springen met spatie en terugvallen.
- Verander uiterlijk of kleur bij actie.

▼ Hulp: Stappen & Scratch-blokken

- `wanneer [toets A] ingedrukt` (Events: "when [key] pressed")
- `wanneer [toets D] ingedrukt` (Events: "when [key] pressed")
- `wanneer [spatie] ingedrukt` (Events: "when [space] key pressed")
- `verander x met [-10]` en `[10]` (Motion: "change x by []")

- verander y met [50] (Motion: "change y by [I]")
- herhaal totdat <raak ik [bodem]?> verander y met -10 (Control: "repeat until <>", Motion: "change y by [I]")
- volgend kostuum (Looks: "next costume")
- zet effect [kleur] op 50 of 0 (Looks: "set [color] effect to [I]")

Previous page
Theorie

Next page
Scratch